UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Ingeniería en Computación

SISTEMAS OPERATIVOS | SEMESTRE 2027-1

Ejercicio de sincronización

El servidor Web

Modelo jefe-trabajador mediante semáforos y variables de condición

Alumno: Francisco Hernández Moguel

Profesor: Gunnar Wolf

Asignatura: Sistemas Operativos

Modalidad: Implementación individual

Fecha: Octubre de 2026

Reporte técnico de implementación, pruebas y refinamiento

Índice

1. Introducción y problema seleccionado **2**
1.1. Planteamiento . . . 2
1.2. Objetivo . . . 2

2. Herramientas y ejecución **2**
2.1. Contenido de la entrega . . . 2
2.2. Instrucciones para ejecutar . . . 2

3. Arquitectura del programa **3**
3.1. Componentes principales . . . 3
3.2. Flujo de las solicitudes . . . 3
3.3. Parámetros de la simulación . . . 3

4. Estrategia de sincronización **4**
4.1. Semáforos: dormir y despertar . . . 4
4.2. Variables de condición y exclusión mutua . . . 4
4.3. Colas seguras y ausencia de espera activa . . . 4

5. Refinamiento: contabilidad del rendimiento **5**
5.1. Datos registrados por trabajador . . . 5
5.2. Notificación antes de finalizar . . . 5
5.3. Cierre ordenado e informe consolidado . . . 5

6. Interfaz gráfica con Tkinter **6**

7. Pruebas de funcionamiento y evidencias **6**
7.1. Prueba 1: estado inicial del servidor . . . 6
7.2. Prueba 2: envío de una solicitud . . . 7
7.3. Prueba 3: cinco solicitudes concurrentes . . . 8
7.4. Prueba 4: saturación con veinte solicitudes . . . 8
7.5. Prueba 5: cierre ordenado e informe de 56 solicitudes . . . 9

8. Uso de inteligencia artificial como apoyo **12**

9. Limitaciones y posibles mejoras **13**

10. Conclusión **13**

Referencias **13**

1. Introducción y problema seleccionado

1.1 Planteamiento

Se eligió el **problema 5, "El servidor Web"**, de la presentación *Ejercicios de sincronización* de la asignatura. Se pide representar un servidor mediante el modelo **jefe-trabajador**: un hilo jefe recibe solicitudes y las asigna a hilos trabajadores. Cada trabajador debe permanecer bloqueado cuando no tenga tareas y despertar al recibir una asignación. Asimismo, el jefe busca mantener disponibles k trabajadores para atender nuevas solicitudes [1].

Alcance de la simulación. El programa no abre sockets, no recibe peticiones HTTP reales y no publica páginas en Internet. Las solicitudes son trabajos simulados identificados por rutas como `/inicio` o `/productos`. Su propósito es estudiar la sincronización entre hilos.

1.2 Objetivo

Desarrollar un programa concurrente en Python que distribuya solicitudes entre trabajadores mediante semáforos, variables de condición y una cola sincronizada. Se incluye una interfaz gráfica para observar el comportamiento del sistema y un refinamiento de contabilidad donde **cada trabajador entrega un informe individual antes de finalizar su ejecución**.

2. Herramientas y ejecución

El programa fue ejecutado y probado en **Windows mediante Visual Studio**, utilizando **Python 3** y las bibliotecas estándar de ese lenguaje: `threading`, `queue`, `time`, `random`, `collections`, `datetime`, `pathlib` y `tkinter` (con `ttk`). No necesita instalar paquetes mediante `pip` si Python incluye Tkinter.

2.1 Contenido de la entrega

- `servidor_web_refinado.py`: código completo del programa.
- `README.md` o este reporte: descripción, ejecución, sincronización, pruebas y refinamiento.
- `informe_servidor.txt`: resultado generado al finalizar el servidor.
- `capturas/`: evidencias gráficas opcionales del proceso de pruebas.

2.2 Instrucciones para ejecutar

En una terminal de Windows, ubicada en la carpeta donde se encuentra el programa:

```
py servidor_web_refinado.py
```

En Ubuntu con interfaz gráfica, puede ser necesario instalar Tkinter primero:

```
sudo apt install python3-tk
python3 servidor_web_refinado.py
```

Después de abrir la ventana, se puede seleccionar una página, enviar una o cinco solicitudes y, al

concluir las pruebas, pulsar **“Finalizar e informar”**. El archivo de resultados se guarda junto al programa.

3. Arquitectura del programa

El diseño separa la sincronización de los elementos visuales, para que las operaciones sobre la interfaz no interfieran directamente con los hilos de trabajo.

3.1 Componentes principales

Componente	Responsabilidad
`Servidor`	Mantiene las colas, el estado compartido, el jefe y los trabajadores.
`_jefe()`	Extrae solicitudes, localiza o crea trabajadores y asigna tareas.
`_trabajador()`	Espera en un semáforo, procesa solicitudes, registra estadísticas y reporta antes de finalizar.
`Interfaz`	Construye la ventana, permite enviar solicitudes y presenta el estado de los hilos.
`construir_informe()`	Consolida los reportes individuales en un archivo legible.

3.2 Flujo de las solicitudes

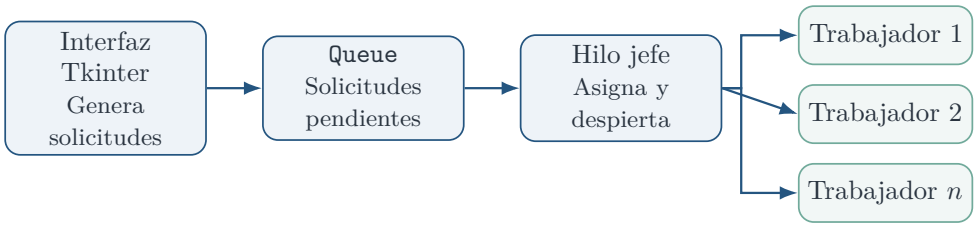


Figura 1: Esquema simplificado de la distribución de solicitudes. El jefe asigna los trabajos a trabajadores coordinados mediante semáforos.

Los resultados y cambios de estado se envían a una segunda cola, la cola de **eventos**. La interfaz la consulta mediante `after()`, de modo que **solo el hilo principal actualiza los widgets** de Tkinter.

3.3 Parámetros de la simulación

Parámetro	Valor y función
`RESERVA`	3; cantidad de trabajadores que el jefe intenta mantener disponibles.
`MAX_TRABAJADORES`	8; límite total para evitar crear hilos sin control.
`DURACION_MIN/MAX`	2.0 y 4.0 segundos; duración aleatoria del procesamiento simulado.
Páginas	`/inicio`, `/productos`, `/contacto` y `/usuarios`.

El límite de ocho trabajadores es una **decisión de diseño**, no un requisito numérico indicado por el profesor. Si se alcanza, las solicitudes adicionales esperan hasta que haya un trabajador disponible.

4. Estrategia de sincronización

La implementación emplea el **patrón de señalización** y lo combina con exclusión mutua, variables de condición y una cola segura para hilos. Estos mecanismos se relacionan con los contenidos de *Primitivas de sincronización* vistos en clase [2].

4.1 Semáforos: dormir y despertar

Cada trabajador tiene un semáforo inicializado en cero:

```
datos = {"semaforo": threading.Semaphore(0)}
```

Cuando no tiene trabajo ejecuta la operación `acquire()`, equivalente a esperar o decrementar el semáforo (operación P). El jefe asigna una tarea y utiliza `release()`, equivalente a señalar (operación V):

```
# Trabajador: se bloquea hasta recibir trabajo.
datos["semaforo"].acquire()

# Jefe: asigna y despierta al trabajador elegido.
datos["tarea"] = tarea
datos["semaforo"].release()
```

Las operaciones se ejecutan en hilos distintos. Se evita consultar continuamente una variable para saber si ya llegó trabajo.

4.2 Variables de condición y exclusión mutua

Cuando no quedan trabajadores libres y se alcanzó el máximo, el jefe espera mediante una variable de condición. Esta libera temporalmente el candado al esperar y vuelve a adquirirlo cuando despierta:

```
with self.condicion:
    while not self.libres:
        if len(self.trabajadores) < MAX_TRABAJADORES:
            self._crear_trabajador()
        else:
            self.condicion.wait()
```

Después de terminar una solicitud, el trabajador informa al jefe que existe capacidad disponible:

```
with self.condicion:
    self.libres.append(numero)
    self.condicion.notify()
```

Los bloques `with self.condicion:` también protegen las estructuras compartidas, como la lista de hilos libres, las tareas asignadas y los contadores.

4.3 Colas seguras y ausencia de espera activa

La cola `solicitudes` utiliza `queue.Queue()`, que sincroniza las operaciones entre los productores y el jefe. Este último espera en `get()` si no existen solicitudes pendientes. La cola `eventos` permite

comunicar resultados a la interfaz.

La instrucción `time.sleep()` se usa **solo para representar el tiempo de atención de una solicitud**; no es el mecanismo de sincronización. Las esperas necesarias se realizan por medio de primitivas bloqueantes.

5. Refinamiento: contabilidad del rendimiento

Refinamiento implementado para solicitar crédito adicional.

La presentación propone el seguimiento del sistema y solicita que *cada hilo notifique su rendimiento antes de terminar su ejecución*, incluyendo información como las páginas solicitadas. En esta implementación, **cada trabajador genera su propio informe final** antes de salir, y el servidor consolida esos informes en un archivo de texto [1].

5.1 Datos registrados por trabajador

Cada hilo mantiene un contador de solicitudes atendidas, un tiempo acumulado de procesamiento y un registro de páginas mediante `Counter`. Los valores se actualizan bajo la condición de sincronización.

```
with self.condicion:
    datos["atendidas"] += 1
    datos["tiempo"] += duracion
    datos["paginas"][pagina] += 1
```

5.2 Notificación antes de finalizar

El procedimiento del trabajador usa un bloque `finally` para entregar un informe que queda almacenado en `self.reportes` y se envía a la cola de eventos de la interfaz:

```
finally:
    with self.condicion:
        informe = {
            "atendidas": datos["atendidas"],
            "tiempo": datos["tiempo"],
            "paginas": dict(datos["paginas"]),
        }
        self.reportes[numero] = informe
        self.eventos.put(("informe_hilo", numero, informe))
```

Esta operación se realiza desde el propio hilo trabajador, **antes de finalizar su ejecución**, por lo que implementa el requisito puntual del refinamiento.

5.3 Cierre ordenado e informe consolidado

Al pulsar **“Finalizar e informar”**, el servidor deja de aceptar solicitudes nuevas y coloca un marcador de cierre al final de la cola. El jefe distribuye los trabajos pendientes y después termina. Los trabajadores concluyen sus tareas, envían su informe y salen. Un hilo auxiliar utiliza `join()`

para esperar su finalización sin bloquear la ventana de Tkinter. Finalmente se construye y guarda `informe_servidor.txt`, con los resultados generales e individuales.

Interpretación de los tiempos. El tiempo total acumulado suma la duración de procesamiento de todos los hilos. Como trabajan de forma concurrente, **no equivale al tiempo transcurrido de reloj** de la prueba.

6. Interfaz gráfica con Tkinter

Para que la concurrencia pudiera observarse fácilmente, se desarrolló una interfaz de escritorio mediante `tkinter` y `ttk`, las herramientas de interfaz gráfica incluidas en Python. Se utilizó una disposición con colores oscuros, botones de control, indicadores de solicitudes y una tabla de hilos.

La interfaz se divide en cuatro áreas: **controles de envío**, **indicadores de estado** (recibidas, en cola, en proceso y terminadas), **tabla de trabajadores** (dormido, procesando o finalizado) e **historial de eventos**. Al finalizar aparece una ventana con el reporte, que además se guarda como archivo de texto.

La lógica concurrente está separada de la presentación. Los hilos publican eventos en `Queue`, y el hilo principal de Tkinter los recupera con `after(80, ...)`. Esto evita que los trabajadores modifiquen widgets desde hilos secundarios.

7. Pruebas de funcionamiento y evidencias

Las siguientes verificaciones se realizaron con la interfaz y se documentan para que puedan repetirse. Las primeras tres pruebas corresponden al comportamiento funcional observado durante el desarrollo; el último caso corresponde al informe entregado por la versión refinada.

7.1 Prueba 1: estado inicial del servidor

Procedimiento: ejecutar `servidor_web_refinado.py` sin enviar solicitudes. Deben crearse tres trabajadores, inicialmente dormidos, y los contadores deben mostrar cero.

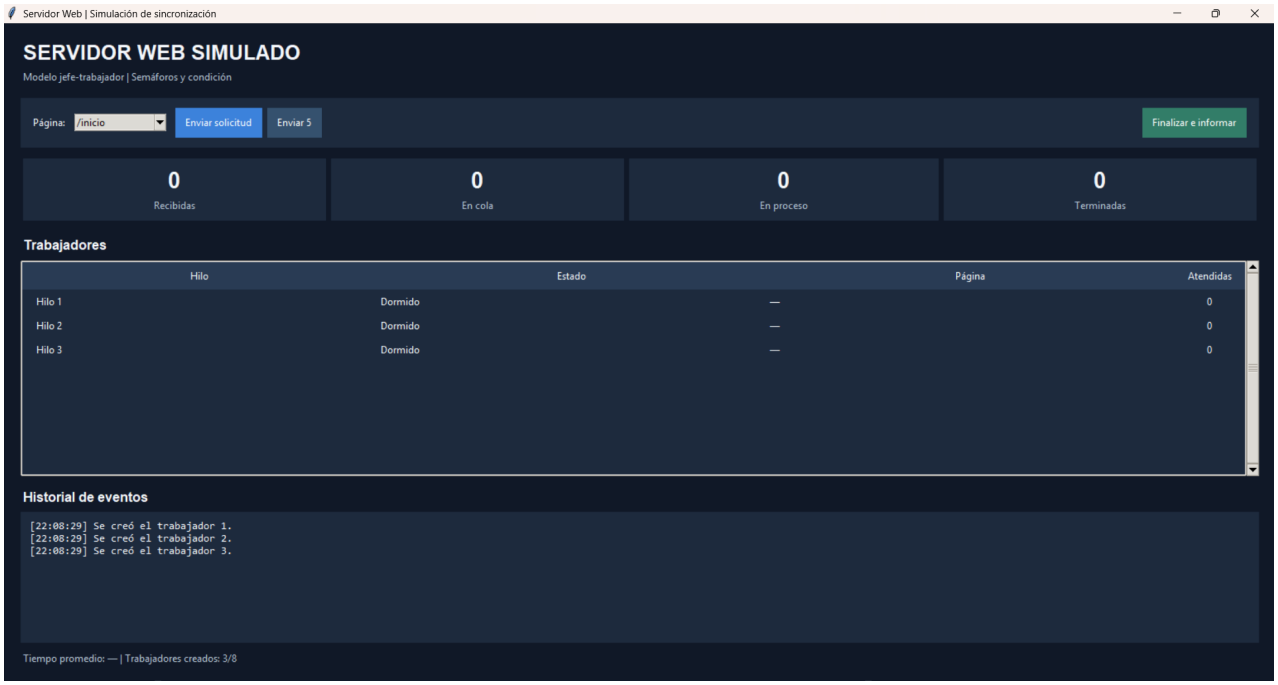


Figura 2: Estado inicial de la interfaz con tres trabajadores disponibles.

7.2 Prueba 2: envío de una solicitud

Procedimiento: seleccionar `/inicio` y pulsar **“Enviar solicitud”**. El jefe despierta a un trabajador, procesa la tarea y crea un cuarto hilo para conservar la reserva de tres libres. Al finalizar: una solicitud recibida y una terminada.

ESPACIO PARA CAPTURA

Subir la imagen y nombrarla:

`.png`

El recuadro desaparecerá automáticamente al compilar con el archivo.

Figura 3: Procesamiento de una solicitud y creación de un trabajador adicional.

7.3 Prueba 3: cinco solicitudes concurrentes

Procedimiento: pulsar una vez **“Enviar 5”**. Se observa la asignación concurrente de solicitudes y la creación de trabajadores hasta alcanzar, cuando es necesario, el máximo de ocho. Al terminar deben coincidir cinco recibidas y cinco terminadas, sin pendientes.

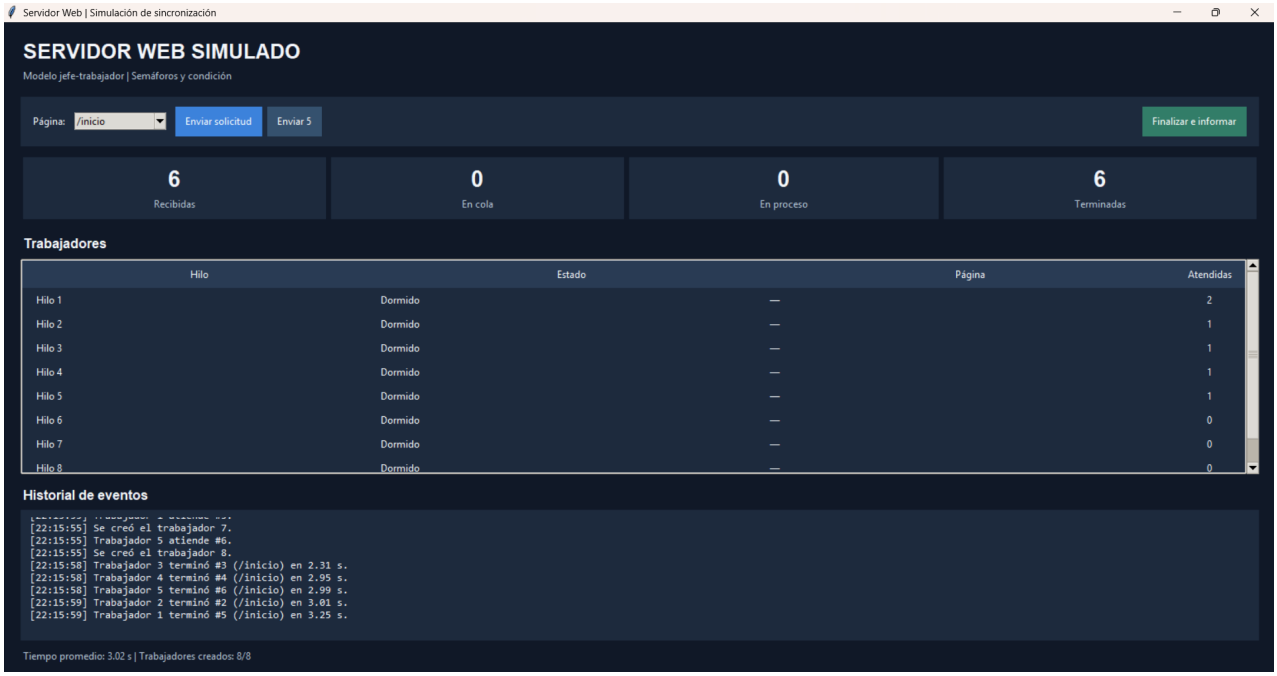


Figura 4: Cinco solicitudes distribuidas entre varios hilos.

7.4 Prueba 4: saturación con veinte solicitudes

Procedimiento: pulsar cuatro veces **“Enviar 5”** de manera consecutiva. Con ocho trabajadores ocupados, las solicitudes restantes permanecen en cola y se procesan conforme se libera capacidad.

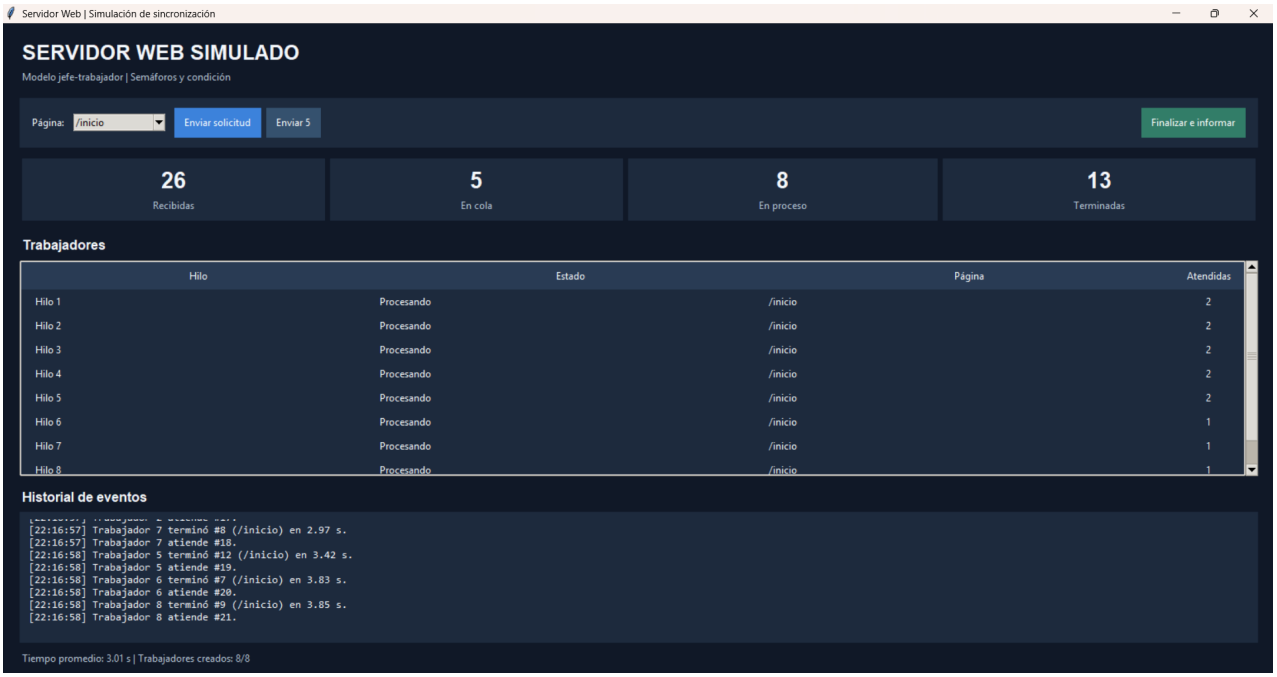


Figura 5: Estado temporal de saturación: solicitudes en cola, en proceso y terminadas.

7.5 Prueba 5: cierre ordenado e informe de 56 solicitudes

Para comprobar el funcionamiento del refinamiento, se realizó una nueva prueba utilizando la versión final del servidor Web simulado. Durante esta ejecución se generaron un total de **56 solicitudes**, distribuidas entre diferentes páginas del servidor.

Una vez enviadas las solicitudes, se utilizó la opción **"Finalizar e informar"**, permitiendo que el servidor terminara de procesar las tareas pendientes y que cada hilo trabajador notificara sus estadísticas individuales antes de finalizar.

Resultados generales

Los resultados obtenidos fueron los siguientes:

Indicador	Resultado observado
Solicitudes recibidas	56
Solicitudes atendidas	56
Trabajadores que finalizaron e informaron	8
Tiempo acumulado de procesamiento	170.42 s
Promedio por solicitud	3.04 s

Se observa que el número de solicitudes recibidas coincide con el número de solicitudes atendidas. Esto indica que todas las tareas aceptadas durante esta ejecución fueron procesadas antes del cierre del servidor.

Además, los ocho trabajadores generaron sus respectivos informes de rendimiento.

Es importante señalar que el tiempo acumulado de 170.42 segundos representa la suma del tiempo de procesamiento de los trabajadores, y no el tiempo real transcurrido, debido a que las solicitudes se atienden concurrentemente.

Distribución de páginas solicitadas

Durante la ejecución se solicitaron las siguientes páginas:

Página	Solicitudes
`/contacto`	5
`/inicio`	46
`/productos`	5
Total	**56**

La página `/inicio` concentró la mayor cantidad de solicitudes, con 46 peticiones. Las páginas `/contacto` y `/productos` recibieron cinco solicitudes cada una.

Las tres páginas fueron atendidas mediante el mismo mecanismo de distribución jefe-trabajador.

Rendimiento individual de los trabajadores

El refinamiento implementado permite registrar el número de solicitudes procesadas por cada hilo, su tiempo acumulado de trabajo y el promedio de procesamiento.

Hilo	Atendidas	Tiempo acum. (s)	Promedio (s)
1	7	24.02	3.43
2	7	21.78	3.11
3	9	23.45	2.61
4	8	23.24	2.90
5	7	22.03	3.15
6	5	16.83	3.37
7	7	20.46	2.92
8	6	18.62	3.10
Total	**56**	**170.42***	**3.04**

* El total procede del informe generado por el programa. La suma de los tiempos individuales mostrados presenta una diferencia de 0.01 s por redondeo.

Páginas atendidas por cada trabajador

La siguiente tabla presenta la distribución de solicitudes que procesó cada hilo durante la ejecución.

Hilo	/contacto	/inicio	/productos	Total
1	1	6	0	7
2	1	6	0	7
3	1	7	1	9
4	1	6	1	8
5	0	6	1	7
6	0	5	0	5
7	1	5	1	7
8	0	5	1	6
Total	**5**	**46**	**5**	**56**

Los resultados muestran que las solicitudes fueron distribuidas entre los ocho trabajadores. El hilo 3 procesó la mayor cantidad, con nueve solicitudes, mientras que el hilo 6 atendió cinco.

Esta diferencia es esperada, ya que los tiempos de procesamiento son variables y los trabajadores vuelven a estar disponibles en distintos momentos.

Evidencias del cierre y del refinamiento

La siguiente captura muestra el estado del servidor durante el cierre ordenado, cuando se permite finalizar las solicitudes pendientes y se recopilan los informes de los trabajadores.

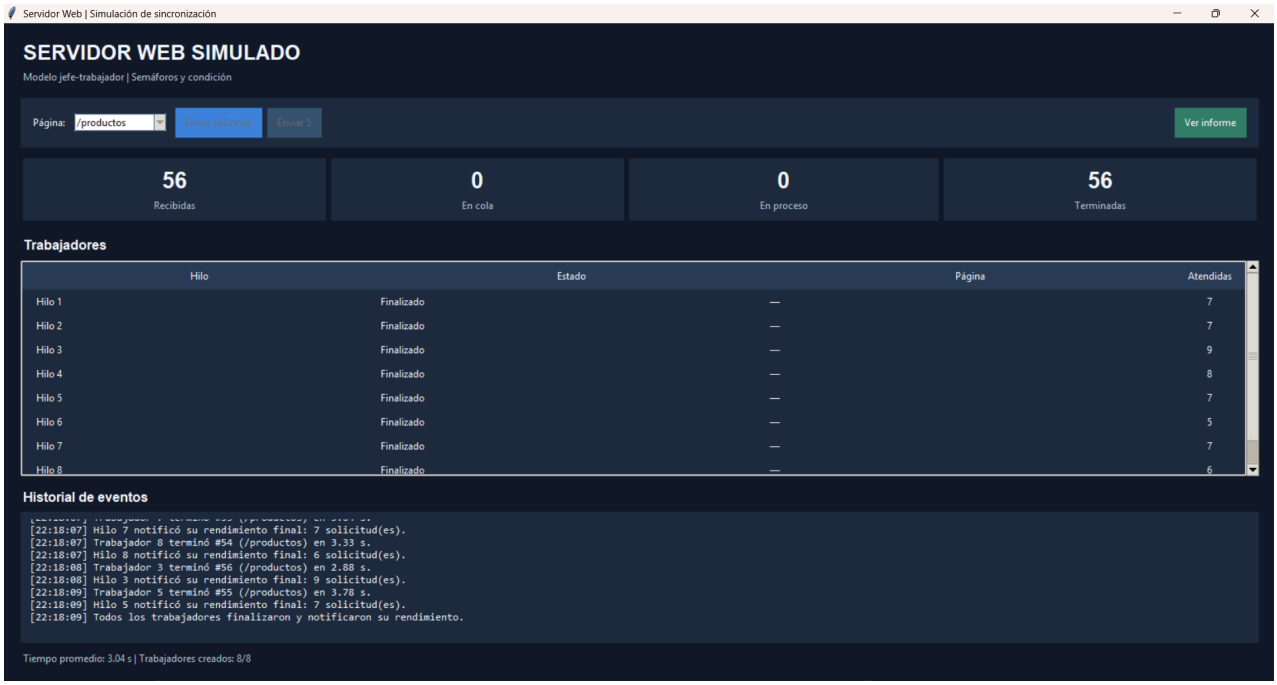


Figura 6: Cierre ordenado del servidor y finalización de los trabajadores.

La siguiente evidencia presenta el informe final generado por el programa, donde se observa la contabilidad general y el rendimiento individual de los hilos.

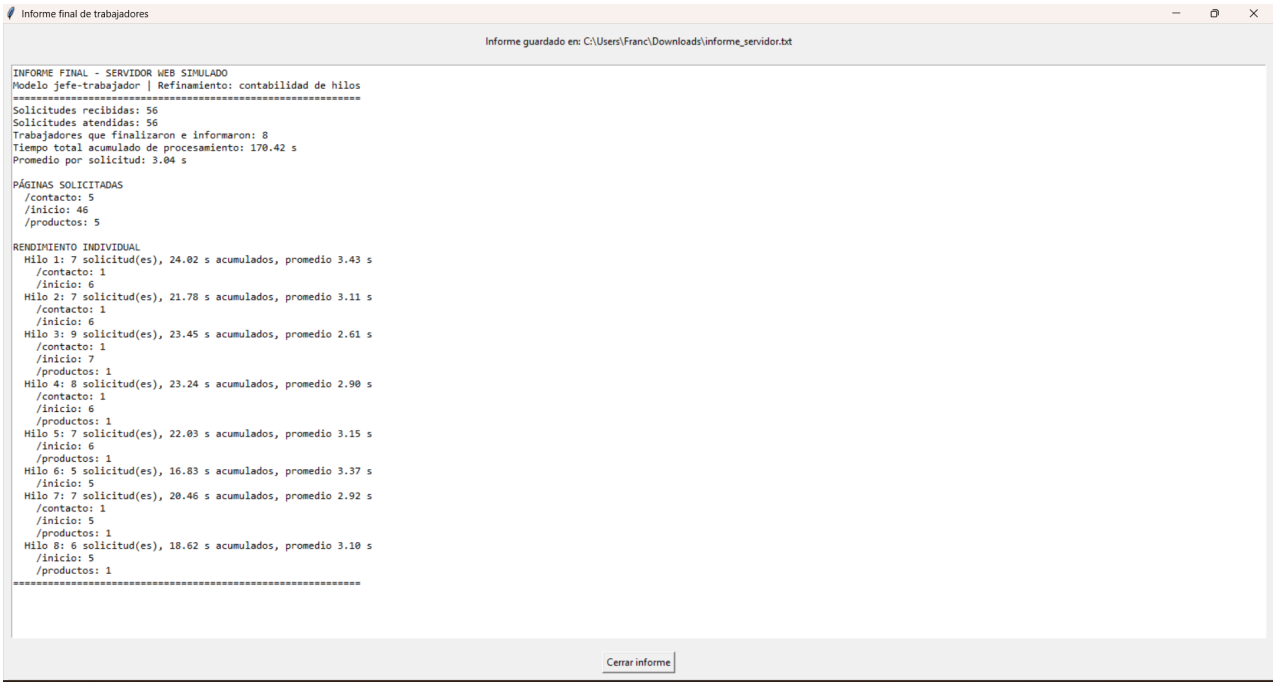


Figura 7: Informe final de 56 solicitudes atendidas y ocho trabajadores.

Verificación del refinamiento:
La ejecución permitió comprobar que los ocho trabajadores entregaron información de rendimiento antes de finalizar.
Los informes individuales incluyen el número de solicitudes atendidas, las páginas procesadas y los tiempos de procesamiento.
La cantidad total de solicitudes recibidas coincide con las atendidas, sin tareas pendientes al finalizar la prueba.
Estos resultados constituyen evidencia funcional de la implementación del refinamiento de seguimiento y contabilidad solicitado por el profesor.

8. Uso de inteligencia artificial como apoyo

Durante el desarrollo se utilizó **ChatGPT como herramienta de asistencia y guía técnica**. Su participación incluyó la discusión de la arquitectura jefe-trabajador, la propuesta y elaboración inicial del código, la explicación de semáforos y variables de condición, la ayuda para diseñar la interfaz gráfica con `Tkinter/ttk`, y la incorporación y revisión del refinamiento de contabilidad.

La IA fue un recurso de apoyo, no un mecanismo de validación: la evidencia de funcionamiento procede de la ejecución del programa, de sus registros y de la explicación de las primitivas de sincronización.

9. Limitaciones y posibles mejoras

- La implementación representa solicitudes simuladas; no acepta conexiones HTTP reales.
- Los tiempos de procesamiento son aleatorios y no reflejan diferencias reales entre páginas.
- La cantidad inicial y máxima de trabajadores se modifica en el código y no desde la interfaz.
- Sería útil realizar pruebas de estrés automatizadas y registrar métricas adicionales, como tiempo de espera en cola y tiempo de respuesta total.
- La corrección bajo todas las intercalaciones posibles de hilos requeriría pruebas y análisis adicionales.

10. Conclusión

El ejercicio permitió implementar y observar un esquema concurrente de tipo **jefe-trabajador**. Los semáforos bloquean a los trabajadores hasta que reciban una tarea; la condición permite al jefe esperar cuando no hay capacidad; y las colas seguras separan la generación de solicitudes de su asignación y del despliegue de eventos.

El **refinamiento de contabilidad** no se limita a mostrar contadores durante la ejecución: cada trabajador entrega, antes de terminar, un informe con sus solicitudes atendidas, tiempos y páginas procesadas. La evidencia de 56 solicitudes atendidas y ocho informes individuales ilustra ese comportamiento. Finalmente, la interfaz en Tkinter facilita visualizar la sincronización y comunicar de forma clara los resultados.

Referencias

Referencias

[1] Wolf, Gunnar. *Administración de procesos: Ejercicios de sincronización*. Material de Sistemas Operativos, Facultad de Ingeniería, UNAM. Apartado “El servidor Web”, pp. 14–16 del material.

[2] Wolf, Gunnar. *Administración de procesos: Primitivas de sincronización*. Material de Sistemas Operativos, Facultad de Ingeniería, UNAM. Secciones de semáforos, variables de condición, señalización y productor-consumidor.

[3] Python Software Foundation. *Python 3 Documentation*. Módulos `threading`, `queue` y `tkinter`. `https://docs.python.org/3/`.